

Chapter 7 | Exercise Answers

More SQL: Queries, Triggers and Views

Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th edition.

Questions 7.1-7.9 follow the photographed chapter supplied for this course. Use the question numbers and printed page locators to read these answers beside your textbook.

These are original worked answers, not an official publisher solution manual. Open-ended designs state their assumptions. Full question text and photographed textbook figures are not reproduced.

Examples are identified as textbook data or original test data. SQL output tables are produced by executing the shown queries. SQLite-specific limitations are stated where they affect the result.

Publication of these answers does not expand the assigned sections or examination scope. Only taught and assigned material is assessed.

7.1. The six query clauses

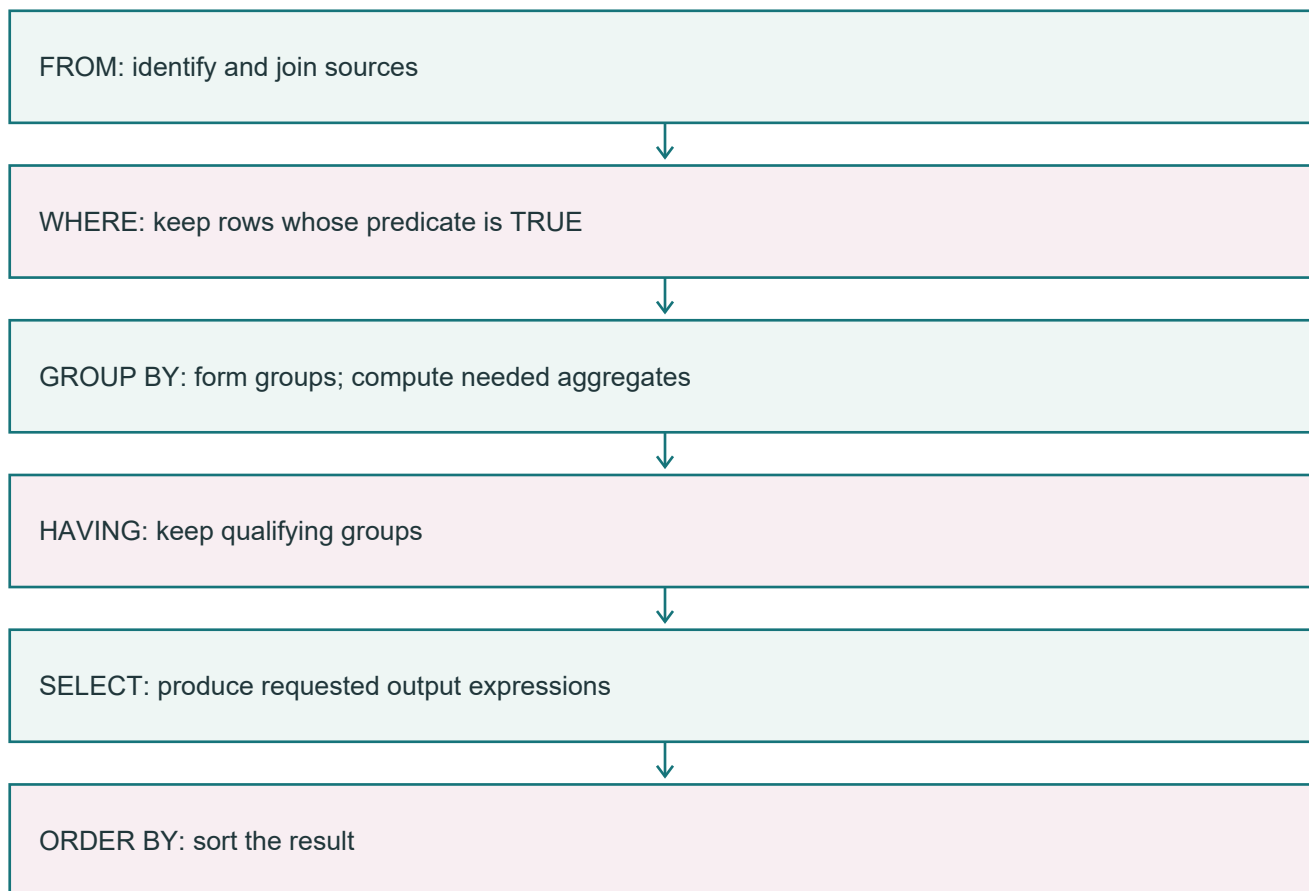
Source: photographed textbook, printed p./pp. 266; explanation 253-254.

Clause	Purpose	Required in the textbook table-query template?
SELECT	Output columns and expressions.	Yes
FROM	Tables, derived tables, joins and aliases.	Yes
WHERE	Filter individual rows.	No
GROUP BY	Partition selected rows by grouping expressions.	No
HAVING	Filter groups using group properties.	No
ORDER BY	Sort the final result.	No

Without GROUP BY, an aggregate query can treat all selected rows as one group. Some dialects allow SELECT of constants without FROM. Do not confuse textual clause order with physical execution order.

7.2. Conceptual query evaluation

Source: photographed textbook, printed p./pp. 266; explanation 254.



This describes query meaning. The optimizer may use indexes, reorder joins or push predicates while preserving that meaning. It need not physically build the entire joined table first.

Example: filter salaries over 30000 before grouping to count highly paid employees. Group first and use `HAVING AVG(Salary)>30000` to choose departments by their overall average. Those two questions are different.

7.3. NULL in predicates, aggregates and groups

Source: photographed textbook, printed p./pp. 266; explanations 238-239, 248-249.

Expression / operation	Result / rule
NULL = NULL	UNKNOWN, not TRUE.
NULL IS NULL	TRUE.
WHERE predicate is UNKNOWN	The row is not selected.
SUM(10, NULL, 20)	30: NULL is ignored; not proof the unknown value equals zero.
COUNT(value) for 10, NULL, 20	2
COUNT(*) for those three rows	3
AVG(value) for 10, NULL, 20	15, not 10.
All values NULL	SUM/AVG/MIN/MAX return NULL; COUNT(value) returns 0.
GROUP BY a nullable column	All NULL grouping values belong to one group.

For (10, NULL, 20), these results were executed in SQLite. Two NULL group values being grouped together does not make ordinary NULL = NULL evaluate to TRUE.

7.4. SQL constructs and useful options

Source: photographed textbook, printed p./pp. 266; explanations 239-265.

Part / construct	Explanation and use
(a) Nested queries	A query inside another statement. IN tests membership; EXISTS tests for a matching row; a scalar subquery supplies one value. Correlated queries reference the outer row. ANY/ALL comparisons are textbook SQL but are not implemented with that syntax in SQLite.
(b) Joined tables	INNER JOIN keeps matches. LEFT/RIGHT/FULL OUTER JOIN also retain unmatched rows from the specified side(s), with NULL placeholders. CROSS JOIN forms combinations. ON makes the matching condition explicit; NATURAL JOIN can unintentionally match newly added same-named columns.
(c) Aggregation and grouping	COUNT/SUM/AVG/MIN/MAX summarize all selected rows or GROUP BY groups. WHERE filters rows; HAVING filters groups. Nonaggregated selected attributes must be compatible with the grouping.
(d) Triggers	Event-condition-action rules run around INSERT/UPDATE/DELETE. BEFORE/AFTER and row/statement options vary by DBMS; SQLite uses row triggers and INSTEAD OF triggers on views. Cover every relevant event, not only the first insertion.
(e) Assertions	A declarative database-wide condition must remain true. A trigger executes an action for specified events. SQLite has no CREATE ASSERTION; an equivalent implementation must handle every change that could violate the condition.
(f) WITH	Names a query result for use in one statement, often called a common table expression. It is not necessarily a stored or materialized table. Recursive forms can express repeated relationships.
(g) CASE	Produces a value conditionally: simple CASE compares one expression; searched CASE checks predicates. ELSE handles the remaining case, otherwise the result is NULL.
(h) Views	A named query presents derived data. A simple single-table view may be updatable in some DBMSs; aggregation or ambiguous joins generally prevent automatic translation. SQLite views are read-only unless INSTEAD OF triggers provide the writes.
(i) Schema changes	CREATE adds an object; ALTER changes supported definitions; DROP removes an object. Dependencies and RESTRICT/CASCADE behavior are dialect-specific. SQLite ALTER TABLE does not implement every textbook ALTER CONSTRAINT form.

```
WITH HIGH_PAY AS (SELECT * FROM EMPLOYEE WHERE Salary>30000)
SELECT Fname,CASE WHEN Salary>=50000 THEN '50k+' ELSE 'under 50k' END
FROM HIGH_PAY ORDER BY Fname;
```

Executed example: Franklin under 50k; James 50k+; Jennifer under 50k; Ramesh under 50k. This is one use of WITH and searched CASE, not a claim that WITH makes the query faster.

Syntax boundaries were checked against SQLite documentation: sqlite.org/lang_createview.html, sqlite.org/lang_createttrigger.html, sqlite.org/lang_altertable.html. Unsupported assertions and ANY/ALL syntax were not reported as successful executions.

7.5. Department aggregates

Source: photographed textbook, printed p./pp. 266; Figure 5.6 p.192.

(a) Group all employees within each department, then test the group average.

```
SELECT D.Dname,COUNT(*) AS Employees
FROM DEPARTMENT D JOIN EMPLOYEE E ON E.Dno=D.Dnumber
GROUP BY D.Dnumber,D.Dname HAVING AVG(E.Salary)>30000
ORDER BY D.Dnumber;
```

Observed result

Dname	Employees
Headquarters	1
Administration	3
Research	4

Department	Salary sum	Count	Average
1	55000	1	55000
4	93000	3	31000
5	133000	4	33250

(b) Yes. Keeping part (a)'s qualifying departments, change only the counted people to males earning more than 30000:

```
SELECT D.Dname,
       SUM(CASE WHEN E.Sex='M' AND E.Salary>30000
                THEN 1 ELSE 0 END) AS Male_employees
FROM DEPARTMENT D JOIN EMPLOYEE E ON E.Dno=D.Dnumber
GROUP BY D.Dnumber,D.Dname HAVING AVG(E.Salary)>30000
ORDER BY D.Dnumber;
```

Observed result

Dname	Male employees
Headquarters	1
Administration	0
Research	2

Administration remains a qualifying department but contributes zero qualifying men. A WHERE filter on male salary would incorrectly remove its other employees before calculating the overall average.

Alternative reading: if (b) replaces the department-average condition entirely, count qualifying men in every department having one. Then use:

```
SELECT D.Dname,COUNT(*) AS Male_employees
FROM DEPARTMENT D JOIN EMPLOYEE E ON E.Dno=D.Dnumber
WHERE E.Sex='M' AND E.Salary>30000
GROUP BY D.Dnumber,D.Dname ORDER BY D.Dnumber;
```

Observed result

Dname	Male employees
Headquarters	1
Research	2

This alternative omits zero-count departments. The wording is not completely explicit, so the two interpretations are not silently treated as identical.

7.6. All A versus no recorded A

Source: photographed textbook, printed p./pp. 267; Figure 2.1 schema p.64.

Return both Name and Major. Original test rows, not Figure 1.2 data, are used below. Assume a straight-A student has at least one registration and a known A in every registered course.

Student	Major	Grades
Alex	MATH	A, A
Blair	CS	B
Casey	MATH	NULL
Drew	MATH	No rows
Emery	CS	A, NULL

```
SELECT S.Name,S.Major FROM STUDENT S
WHERE EXISTS (SELECT 1 FROM GRADE_REPORT G
              WHERE G.Student_number=S.Student_number)
AND NOT EXISTS (SELECT 1 FROM GRADE_REPORT G
               WHERE G.Student_number=S.Student_number
                 AND (G.Grade<>'A' OR G.Grade IS NULL))
ORDER BY S.Student_number;
```

Observed result

Name	Major
Alex	MATH

The EXISTS clause excludes students with no courses. The NULL test prevents an unknown grade from being accepted as an A. Without EXISTS, a student with no courses satisfies "no counterexample" by vacuous truth.

```
SELECT S.Name,S.Major FROM STUDENT S
WHERE NOT EXISTS (SELECT 1 FROM GRADE_REPORT G
                  WHERE G.Student_number=S.Student_number AND G.Grade='A')
ORDER BY S.Student_number;
```

Observed result

Name	Major
Blair	CS
Casey	MATH
Drew	MATH

(b) means no A is recorded: Blair, Casey and Drew qualify. It does not prove that Casey's unknown grade is not A. If the intended question requires known completed grades, add a separate completion condition; do not let NULL choose the policy accidentally.

7.7. Three employee queries

Source: photographed textbook, printed p./pp. 267; Figure 5.6 p.192.

(a) IN allows several tied highest-paid employees in different departments.

```
SELECT Fname,Lname FROM EMPLOYEE
WHERE Dno IN (SELECT Dno FROM EMPLOYEE
              WHERE Salary=(SELECT MAX(Salary) FROM EMPLOYEE))
ORDER BY Fname;
```

Observed result

Fname	Lname
James	Borg

(b) Follow exactly two supervision links, not every descendant.

```
SELECT E.Fname,E.Lname
FROM EMPLOYEE E JOIN EMPLOYEE S ON E.Super_ssn=S.Ssn
WHERE S.Super_ssn='888665555' ORDER BY E.Fname;
```

Observed result

Fname	Lname
Ahmad	Jabbar
Alicia	Zelaya
John	Smith
Joyce	English
Ramesh	Narayan

(c) The minimum is 25000, so the inclusive threshold is 35000.

```
SELECT Fname,Lname FROM EMPLOYEE
WHERE Salary >= (SELECT MIN(Salary)+10000 FROM EMPLOYEE)
ORDER BY Fname;
```

Observed result

Fname	Lname
Franklin	Wong
James	Borg
Jennifer	Wallace
Ramesh	Narayan

7.8. Four useful views

Source: photographed textbook, printed p./pp. 267; Figure 5.6 p.192.

(a) LEFT JOIN retains a department even if its optional manager reference is NULL.

```
CREATE VIEW DEPT_MANAGER AS
SELECT D.Dnumber,D.Dname,M.Fname,M.Lname,M.Salary
FROM DEPARTMENT D LEFT JOIN EMPLOYEE M ON M.Ssn=D.Mgr_ssn;
```

Observed result

Dnumber	Dname	Fname	Lname	Salary
1	Headquarters	James	Borg	55000
4	Administration	Jennifer	Wallace	43000
5	Research	Franklin	Wong	40000

(b) Use different aliases for employee and supervisor, retaining an employee without a supervisor.

```
CREATE VIEW RESEARCH_STAFF AS
SELECT E.Ssn,E.Fname,E.Lname,S.Fname AS Supervisor_first,
       S.Lname AS Supervisor_last,E.Salary
FROM EMPLOYEE E JOIN DEPARTMENT D ON D.Dnumber=E.Dno
LEFT JOIN EMPLOYEE S ON S.Ssn=E.Super_ssn
WHERE D.Dname='Research';
```

Observed result

Ssn	Fname	Lname	Supervisor first	Supervisor last	Salary
123456789	John	Smith	Franklin	Wong	30000
333445555	Franklin	Wong	James	Borg	40000
453453453	Joyce	English	Franklin	Wong	25000
666884444	Ramesh	Narayan	Franklin	Wong	38000

(c) Count non-NULL worker keys so an empty project counts as zero, not one.

```
CREATE VIEW PROJECT_SUMMARY AS
SELECT P.Pnumber,P.Pname,D.Dname,COUNT(W.Essn) AS Employees,
       SUM(W.Hours) AS Known_hours
FROM PROJECT P JOIN DEPARTMENT D ON D.Dnumber=P.Dnum
LEFT JOIN WORKS_ON W ON W.Pno=P.Pnumber
GROUP BY P.Pnumber,P.Pname,D.Dname;
```

Observed result

Pnumber	Pname	Dname	Employees	Known hours
1	ProductX	Research	2	52.5
2	ProductY	Research	3	37.5
3	ProductZ	Research	2	50
10	Computerization	Administration	3	55
20	Reorganization	Headquarters	3	25
30	Newbenefits	Administration	3	55

Known_hours is SQL SUM(Hours). Reorganization has 25 known hours plus Borg's unknown hours. Its complete total is therefore unknown, not necessarily 25. For a project with no known Hours values, SUM is NULL; this is not silently changed to zero.

(d) Filter the aggregate view; equivalently use HAVING COUNT(W.Essn)>1 in (c).

```
CREATE VIEW SHARED_PROJECTS AS
SELECT * FROM PROJECT_SUMMARY WHERE Employees>1;
```

Observed result

Pnumber	Pname	Dname	Employees	Known hours
1	ProductX	Research	2	52.5
2	ProductY	Research	3	37.5
3	ProductZ	Research	2	50
10	Computerization	Administration	3	55
20	Reorganization	Headquarters	3	25
30	Newbenefits	Administration	3	55

Every project in this particular state has more than one worker, so (c) and (d) show the same rows. That equality is a property of this state, not of the view definitions.

7.9. Querying and attempting to update a grouped view

Source: photographed textbook, printed p./pp. 267-268; Figure 5.6 p.192.

Use the photographed grouped-view definition:

```
CREATE VIEW DEPT_SUMMARY(D,C,Total_s,Average_s) AS
SELECT Dno,COUNT(*),SUM(Salary),AVG(Salary)
FROM EMPLOYEE GROUP BY Dno;
```

(a) Reading all rows is allowed. Equivalent base-table query:

```
SELECT Dno AS D,COUNT(*) AS C,SUM(Salary) AS Total_s,
       AVG(Salary) AS Average_s
FROM EMPLOYEE GROUP BY Dno ORDER BY Dno;
```

Observed result

D	C	Total s	Average s
1	1	55000	55000
4	3	93000	31000
5	4	133000	33250

(b) Reading groups whose salary total exceeds 100000 is allowed:

```
SELECT Dno AS D,COUNT(*) AS C FROM EMPLOYEE
GROUP BY Dno HAVING SUM(Salary)>100000 ORDER BY Dno;
```

Observed result

D	C
5	4

(c) Department 4 has three employees. Reading departments with more employees is allowed:

```
SELECT Dno AS D,AVG(Salary) AS Average_s FROM EMPLOYEE
GROUP BY Dno HAVING COUNT(*) >
  (SELECT COUNT(*) FROM EMPLOYEE WHERE Dno=4)
ORDER BY Dno;
```

Observed result

D	Average s
5	33250

The equivalent query in (c) assumes department 4 has rows, as it does in Figure 5.6. If it has none, the original scalar subquery over DEPT_SUMMARY returns NULL, whereas COUNT(*) over an empty base set returns zero; preserve that distinction for another state.

Part	Allowed as a direct grouped-view write?	Reason
(d) UPDATE D=4 to D=3	No	The grouped view is not automatically updatable. A separate business operation could move employees, but no department 3 exists in this state, so that proposed base update would also violate its FK.
(e) DELETE rows with C>4	No	The view is not directly deletable. No displayed department has more than four employees, but a zero-match predicate does not make an otherwise unsupported view write legal.

SQLite tests reject both direct writes. INSTEAD OF triggers could define custom write semantics, but none are supplied and their behavior must not be invented.